# Detect, Restart, Abstain

Cheap mitigation of LLM failures and hallucinations,
and why fixing the choke point does not pay

## The research story, start to finish

and a guide to the MSE-2 evaluation

Harsh Dubey  Harshit Saini  Prashant Jain
Himanshu Kumar  Gaurav Kaushal  Rohit Chaudhary (supervisor)

Department of CSE (AI&ML), KIET Group of Institutions, Ghaziabad

**How to use this document.** It tells what the project did, why, and what came out, in the order it happened. Each part of the MSE-2 evaluation has its own section, so you can open the one being asked about:

| MSE-2 criterion | Where | Marks |
|---|---|---|
| Project: Model Training | Section 5 | 5 |
| Project: Hyperparameter Tuning | Section 6 | 5 |
| Project: Result Analysis | Section 8 | 5 |
| Research paper: Methodology | Section 4 | 2.5 |
| Research paper: Results | Section 7 | 2.5 |

Every number in this guide is printed from the same generated file as the paper (`paper/values.tex`, written by `paper/make_values.py` from the result tables), so the guide and the paper cannot disagree. Blue boxes give the one-line answer to say in the viva.

Status: preprint draft, October 2026. Code and data:
`github.com/HarshDubey23/early-failure-detection-cot`

# Contents

**1 The whole project in one page** **3**

**2 How the project unfolded** **4**

**3 Background in plain words** **5**

**4 Methodology** **5**

4.1 Data . . . 6
4.2 Part 1: is early warning a difficulty signal? . . . 6
4.3 Part 2: counterfactual commitment . . . 6
4.4 Recovery policies at matched compute . . . 7
4.5 ChokeBench (new) . . . 7
4.6 Part 3: recovery from hallucinations . . . 7
4.7 Statistics and evidence levels . . . 7
4.8 Theory: three propositions . . . 7

**5 Model training** **8**

5.1 What is frozen and what is trained . . . 8
5.2 How the failure detectors are trained (Parts 1 and E3) . . . 9
5.3 How the learned restart-point predictor (P4) is trained . . . 9
5.4 How the learned choke-point localiser is trained (new) . . . 9

**6 Hyperparameter tuning** **10**

6.1 Principle: tune only where the test set cannot leak in . . . 10
6.2 1. Fixed in advance, then checked for stability (detectors) . . . 10
6.3 2. Measurement hyperparameters: varied as sensitivity analyses . . . 10
6.4 3. Tuned with nested cross-validation (learned localiser) . . . 10
6.5 4. A threshold tuned on DEV, tested on TEST (judge) . . . 11

**7 Results** **12**

7.1 Mitigation: what works and what it costs . . . 12
7.2 Part 1: early warning is a difficulty signal . . . 13
7.3 Part 2: where failures are decided . . . 13
7.4 Recovery policies at matched compute . . . 13
7.5 ChokeBench . . . 14
7.6 Part 3: hallucination recovery . . . 14

**8 Result analysis: what the numbers mean** **14**

8.1 Reading 1: why early warning looked good and is not . . . 15
8.2 Reading 2: errors are not choke points . . . 15
8.3 Reading 3: verifiers learn the label they are trained on . . . 15

8.4 Reading 4: why restart wins . . . . . 15
8.5 Reading 5: hallucinations behave the same way, and graders can fool you . . . . . 15
8.6 Reading 6: what to do in practice . . . . . 15

**9 Mistakes we caught and corrected** **16**

**10 Limitations and next steps** **16**

**11 Viva questions with short answers** **16**

# 1 The whole project in one page

**The goal.** Reduce the failures of large language models (LLMs): wrong answers in step-by-step maths and hallucinations (claims not supported by the source) in retrieval-augmented generation (RAG), at the lowest possible cost.

**The popular idea we tested.** Most methods are *surgical*: find the step where the output went wrong, go back to just before it, and repair or regenerate from there. Process reward models (PRMs) are trained to find that step; hallucination-repair systems do the same for unsupported sentences. It sounds efficient, because it keeps the good part of the answer.

**The one quantity.** For a solution written by model $M$, let

$$v_k = P_M(\text{correct final answer} \mid \text{question and the first } k \text{ steps}).$$

We estimate $v_k$ by letting $M$ itself continue from step $k$ many times and counting how often it ends up correct. $v_0$ is "solve from scratch". Resampling from $k = 0$ is a *restart*; resampling from just before step $j$ is *backtracking*. The step where $v$ collapses is the *choke point*; the step a human marks as the first mistake is the *error*.

**What we found: the mitigation that works.**

1. **Detect, restart, abstain (DRA) is the cheapest mitigation that works.** Plain restarting recovers 44–66% of failed maths solutions within four attempts' compute (56–77% within eight). In RAG, a detector-gated resampling loop cuts hallucination by -0.047 (preregistered, $p = 0.0003$), and plain DRA does the same at 1.36× the base cost, cheaper than claim repair (1.71×), our choke-point-aware controller (1.77×) and Chain-of-Verification (3.75×).
2. **A law, with proof.** A restart loop delivers answers whose hallucination rate equals the detector's $P(\text{hallucinated} \mid \text{pass})$, whatever the retry budget. With human labels this predicts 50.2% → 26.6% hallucinated answers.

**What we found: why the surgical version does not pay.**

3. Early-warning detectors mostly measure how hard the question is: question and model alone reach AUROC 0.588; the first quarter of the reasoning adds +0.005 (not significant).
4. 33–66% of failures are decided before the first step. When the outcome is still open, the human-marked error is the choke point in 0.438–0.647 of traces, against a chance level of about 0.18 (preregistered, confirmed on Llama-3.1-8B).
5. A PRM finds the human-marked error 73% of the time but the choke point only 45% (our new benchmark, ChokeBench). A model we trained and tuned for the choke point does no better (0.39).
6. No deployable choice of where to resample reliably beats a plain restart at equal compute. Even a free, perfect choke-point oracle gains only +0.028 at four restarts' budget; in RAG, choosing where to resample adds -0.000.
7. The measured benefit of a detector-gated loop depends on who grades it: +0.326 by a grader trained like the detector, +0.017 (n.s.) by an independent one.

**If you get one sentence**

"To reduce LLM failures and hallucinations cheaply, detect them and restart, abstaining if still unsafe; we prove the result is only as good as the detector, and show by resampling the model itself that the popular alternative, finding the bad step and repairing from there, does not beat restarting at equal compute, because failures are rarely decided where we can see them."

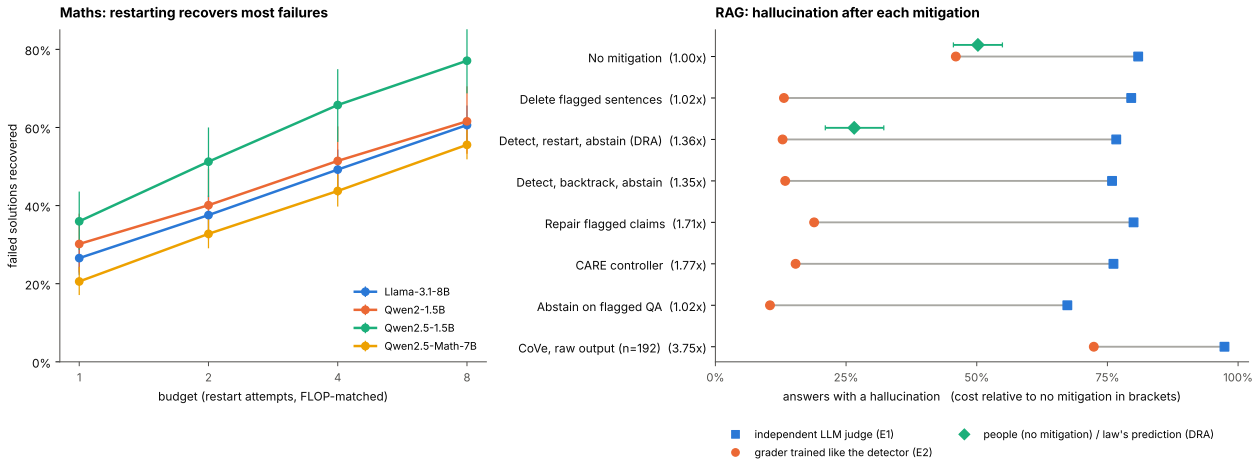



Figure 1: What mitigation buys: failed maths solutions recovered by restarting (left) and hallucination after each RAG mitigation, with its cost (right).

## 2 How the project unfolded

Research rarely goes in a straight line. This one changed direction twice, and both changes made it stronger. The dates come from the git history.

| When | What happened and why |
|---|---|
| 15 Sep 2026 | **Original study (E1–E8).** Question: can a partial chain of thought predict a wrong final answer, and where do first errors occur? Data: ProcessBench (3,400 maths solutions from 12 models, with the expert-marked first wrong step). We trained detectors (TF-IDF + logistic regression, a surface-feature MLP and a GRU) on growing prefixes of each solution. Result: the lexical detector reached AUROC 0.588 at 25% of the chain and 0.626 on the whole chain. |
| 24 Oct | **Novelty audit and first pivot.** A literature check showed two problems: (i) other groups had already predicted success from hidden states, and (ii) a detector that never reads the reasoning (question text + which model wrote it) also reaches 0.588. So the "early warning" was mostly a *difficulty* signal. We added difficulty controls and asked a sharper question: is the human-marked first error the point where the outcome is decided? |
| 24 Oct | **Runs A and B (GPU, Kaggle).** We built the counterfactual pipeline: for every prefix of a solution, sample the same model many times and estimate $v_k$. Pilot (Run A) checked noise; Run B covered three Qwen models. The hypotheses H1–H4 were written down and committed *before* any rollout (commit `a6fb04c`). |
| 24–25 Oct | **Preregistered confirmatory Run C.** Before collecting new data we committed a second plan (`459ff22`): tests C1–C4 on a new model family (Llama-3.1-8B), replication R1–R3 on fresh sampling batches, and practical policies with full compute accounting. Some tests were confirmed and some failed; both are reported. |
| 25–26 Oct | **Run R: hallucinations.** We asked whether the same holds for RAG. We built a recovery controller (CARE) that detects a hallucinated sentence and chooses between repairing it, backtracking and restarting by estimated recoverability, preregistered its test, and ran it on RAGTruth and HotpotQA with Llama-2-7B. Two bugs found during the run (a chat preamble corrupting repairs, and CoVe echoing its verification block) were fixed and documented. |
| 26 Oct | **A wrong turn.** A "dual consensus judge" and a "1,500-item generalisation" table were added in a hurry. They looked impressive but were not sound (Section 9). |

| | |
|---|---|
| 27 Oct | **Corrections and the final contributions.** We withdrew those claims, rebuilt the paper from the tables, fixed the references, and added three CPU analyses on the existing data: ChokeBench, a learned and tuned choke-point localiser, and a five-grader robustness check of Part 3. This is the version in the final draft. |
| 27 Oct (evening) | **Reframed around mitigation.** The results answer the original goal, reducing failures and hallucinations, more directly than the title said: detect, restart and abstain is the cheapest mitigation that works, and we added its law with a proof and a cost comparison of every mitigation. The paper is now titled *Detect, Restart, Abstain*. |

**Why the pivot matters (good viva point)**

The first version answered an easier question with a confound we had not controlled. Adding a question-only baseline exposed it. Turning a weak result into a sharp new question, and preregistering the test of it, is what turned a course project into a research paper.

## 3 Background in plain words

**Chain of thought (CoT).** The model writes its reasoning as numbered steps before the answer.

**Process reward model (PRM).** A second model that scores each step as right or wrong. We use Qwen2.5-Math-PRM-7B.

**First error $e$.** The first step a human expert marks as wrong (ProcessBench labels).

**$v_k$ and the choke point $d$.** $v_k$ is defined above; $d = \arg\max_k(v_{k-1} - v_k)$, the step with the largest fall in the chance of success.

**Capability-limited.** The model solves the question from scratch at most one time in eight ($v_0 \le \delta = 1/8$). Then no step is a choke point: the failure was decided before the first step.

**Restart vs backtrack.** Start again from the question, or resample from just before step $j$.

**Matched compute.** Every method gets the same token budget, counted in "restart attempts", including the cost of running the verifier.

**AUROC.** Probability that a detector ranks a random failing solution above a random correct one. 0.5 is chance, 1.0 is perfect.

**Cohen's $\kappa$.** Agreement between two raters beyond chance. 0 is chance, 1 is perfect.

**Bootstrap CI.** Resample the data with replacement many times and read the 2.5th and 97.5th percentiles of the estimate. We resample whole questions or traces, not rows (cluster bootstrap).

**Preregistration.** Writing the hypotheses and analysis into the git history before the data exist, so results cannot be cherry-picked afterwards.

**RAG.** Retrieval-augmented generation: the model answers from given source passages. A *hallucination* is a sentence the source does not support.

## 4 Methodology

**Research paper: Methodology (2.5 marks)**

Three studies connected by one measurement, $v_k$. Data with human error labels; on-policy resampling to measure where outcomes are decided; matched-compute comparison of recovery policies; preregistered tests with cluster-bootstrap CIs; every number generated by code.

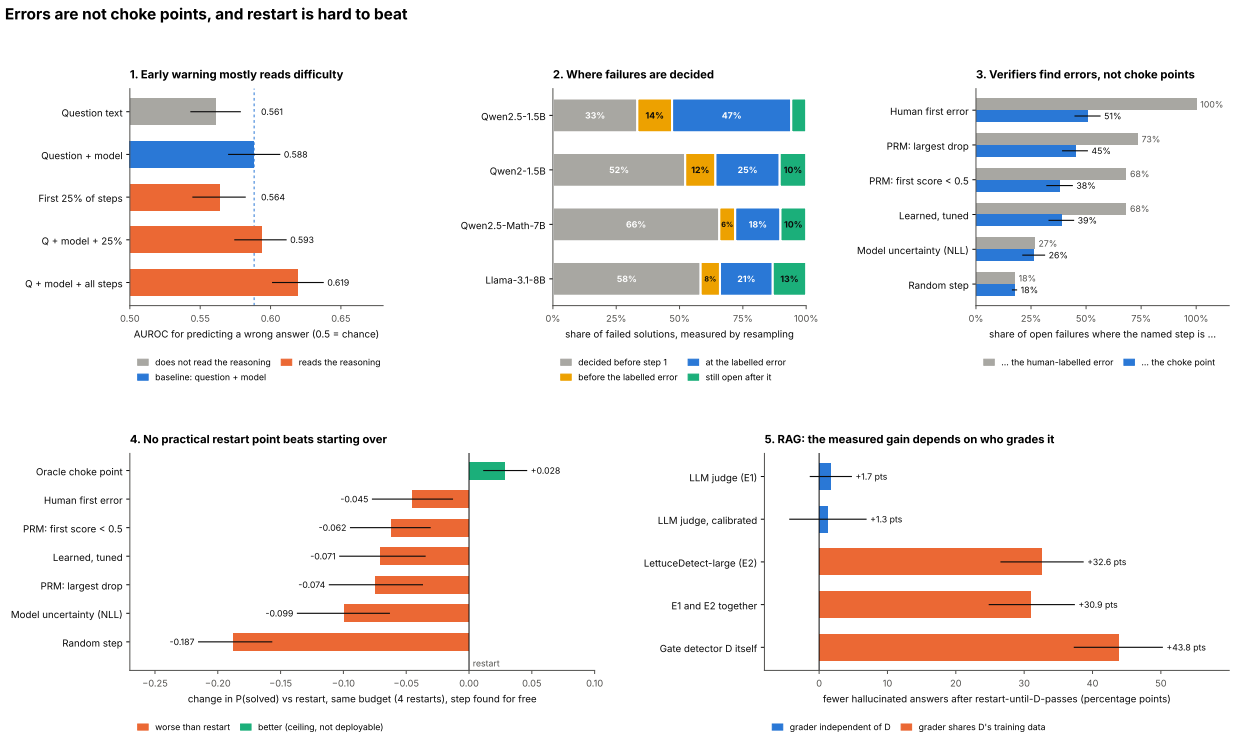



Figure 2: Why the surgical version does not pay. Each panel is one of findings 3–7.

## 4.1 Data

- **ProcessBench**: 3,400 step-by-step solutions to GSM8K, MATH, OlympiadBench and Omni-MATH problems (2,842 distinct problems), written by 12 open models, each with the expert first error and whether the final answer is correct. Correct and incorrect answers are balanced.
- **RAGTruth** test sources (450): Llama-2-7B-chat answers for QA, summarisation and data-to-text, with human-marked hallucinated spans. **HotpotQA**: 300 items with controlled evidence (half have the answer removed, to test abstention).

## 4.2 Part 1: is early warning a difficulty signal?

Detectors read either the question text (Q), the generator and tier (G), a prefix of the reasoning, or combinations. TF-IDF (uni- and bigrams) or one-hot features feed a logistic regression, *cross-fitted over 5 folds grouped by problem* (a problem never appears in both training and test) and repeated over 5 seeds. We report AUROC, the *increment* of the reasoning over Q+G, and a *within-problem* concordance: for problems that have both a failing and a succeeding trace, how often the detector ranks the failing one higher. Within a problem, difficulty is held fixed.

## 4.3 Part 2: counterfactual commitment

For every prefix $s_{1:k}$ ($k = 0, \ldots, n-1$) we sample $m$ continuations from the *same model that wrote the trace* (on-policy) with vLLM at temperature 0.7 and top-$p$ 0.95, and check the final answer against the gold answer with `math-verify`. $m = 64$ for the 1.5B models (four independent batches of 16), 12 for Qwen2.5-Math-7B and 16 for Llama-3.1-8B: 125,832 continuations in total. Each failed trace is then classified as capability-limited, latent commitment (collapse before the marked error), commits at the error, or recoverable error. Three safeguards:

- **Position-matched null**: how often a randomly placed "error" would coincide with the choke point, given each trace's length.
- **Sample splitting**: one half of the continuations chooses the choke point, the other half scores it, over 50 random splits; otherwise noise would be rewarded.
- **Reliability**: correlation of $v_k$ between independent batches, and a Bayesian (Jeffreys) posterior over the choke point (2,000 draws).

### 4.4 Recovery policies at matched compute

Restart; PRM first step below 0.5; PRM largest drop; probe search; a learned restart point; the human first error and the oracle choke point as references. Cost counts every token, weighted by model size (FLOPs $\propto$ parameters $\times$ tokens): prefill once per restart point, every generated token, every PRM pass and probe. With budget $b$ restart attempts, a policy spends what is left after its overhead on independent attempts from its restart point; we report P(at least one correct).

### 4.5 ChokeBench (new)

A benchmark that grades a step localiser on two targets: the human first error and the choke point, and on what the named step is worth as a resampling point (per attempt, and at a $4\times$ budget with localisation free, which upper-bounds any verifier).

### 4.6 Part 3: recovery from hallucinations

The controller CARE gates on detector $D$ (LettuceDetect-base). When $D$ flags a sentence: repair the claim, then resample, choosing restart or backtrack by Thompson sampling on each boundary's recoverability per token, then abstain (QA) or delete what is still flagged. Every policy replays the same bank of continuations under a budget cap of 2. The primary grader $E_1$ is an LLM judge from another family (Qwen2.5-7B-Instruct), validated against RAGTruth's human labels before reading any comparison; $E_2$ (LettuceDetect-large) is secondary.

### 4.7 Statistics and evidence levels

95% cluster-bootstrap CIs (1,000–2,000 resamples); paired differences on the same items; sign-flip permutation p-values with Holm correction for multiple tests; TOST for equivalence ("no difference" is never written as "equal" unless TOST passes). Every claim carries a label: PRE (preregistered), EXP (exploratory), CONF (preregistered test on new data).

### 4.8 Theory: three propositions

**Proposition 1 (a failed prefix is worth no more than a fresh start).** Under the model's own sampling, $v_k$ is a martingale (on average it neither rises nor falls). For any rule that picks the restart point by reading the trace only up to that point,

$$\mathbb{E}[v_\tau \mid \text{fail}] = \frac{\mathbb{E}[v_\tau(1-v_\tau)]}{1-v_0} \le v_0,$$

because $x(1-x)$ is concave. In words: averaged over failures, a failed trace cut at any point chosen without looking ahead is worth no more than a fresh start. Backtracking can only win by looking ahead (e.g. resampling from *before* a flagged step) or by saving tokens.

**Proposition 2 (shorter attempts are the only advantage).** If resampling from point $k$ succeeds no more often per attempt than restarting ($v_k \le v_0$), then with any budget it is worth at most a restart given $c_0/c_k$

times the budget, where $c$ is the length of one attempt. Measured lengths give $c_k/c_0 \approx 0.7$–$0.8$, so the best such point is worth at most 1.25–1.4 restarts' budgets, before paying to find it.

**Proposition 3 (a restart loop is exactly as faithful as its detector).** Suppose the system draws independent answers and returns the first one the detector passes, abstaining after $K+1$ flagged draws. With $h$ the hallucination rate of a draw, $\beta$ the detector's miss rate and $\alpha$ its false-alarm rate,

$$P(\text{hallucinated} \mid \text{answered}) = \frac{h\beta}{h\beta + (1-h)(1-\alpha)}, \qquad P(\text{answered}) = 1 - q^{K+1},\ q = h(1-\beta) + (1-h)\alpha.$$

*Why:* every draw is independently "passed and hallucinated" ($h\beta$), "passed and faithful" ($(1-h)(1-\alpha)$) or "flagged" ($q$); the loop returns the first passed draw, so whichever draw it returns, the odds of hallucinated vs faithful are $h\beta : (1-h)(1-\alpha)$, the same at every try. So more retries buy coverage, never faithfulness, and the place where we resample does not appear in the formula at all. Plugging in the detector's error rates measured on RAGTruth's human labels ($\beta = 28.4\%$, $\alpha = 20.6\%$, $h = 50.2\%$) predicts 26.6% [21.0, 32.2] hallucinated answers after DRA, a reduction of 23.7 points. Two assumptions carry this (independent draws; the detector's errors on resampled answers equal those on drafts), so it is an exploratory estimate until people rate the outputs.

# 5 Model training

**Project: Model Training (5 marks)**

The large language models are used frozen; what we train are the decision models built on top of them: failure detectors, a restart-point predictor and a choke-point localiser, always with group-wise splits so no problem (or no model) leaks from training into testing.

## 5.1 What is frozen and what is trained

| Model | Role | Trained by us? |
|---|---|---|
| Qwen2.5-1.5B, Qwen2-1.5B, Qwen2.5-Math-7B, Llama-3.1-8B (instruct) | generators whose own continuations define $v_k$ | No: frozen. Training them would change the very thing we measure. |
| Llama-2-7B-chat | RAG generator | No (frozen) |
| Qwen2.5-Math-PRM-7B | step verifier | No; re-validated: ProcessBench F1 73.2 vs published 73.5 |
| LettuceDetect base / large | hallucination detector / grader | No (frozen, trained by its authors on RAGTruth) |
| Qwen2.5-7B-Instruct | LLM judge $E_1$ | No; validated against human labels ($\kappa = 0.219$) |
| TF-IDF + logistic regression | failure detectors (Parts 1, E3) | **Yes** |
| Surface-feature MLP (64–32) and GRU (hidden 32) | failure detectors (E3) | **Yes** |
| Stepwise lexical detectors | one per step count, for lead time (E5) | **Yes** |
| Learned restart-point predictor (P4) | picks where to resample | **Yes** |
| Learned choke-point localiser | ChokeBench entry | **Yes** (with tuning, Section 6) |
| Calibrated judge threshold | D-independent grader for Part 3 | **Yes** (threshold fit on DEV) |

### 5.2 How the failure detectors are trained (Parts 1 and E3)

1. **Input.** A prefix of the solution: the first $k = \max(1, \lceil fn \rceil)$ steps for $f$ in a grid from 10% to 100%; or the question text; or the generator's identity.
2. **Features.** TF-IDF of words and word pairs (at most 20,000 features, a term must appear in at least 2 training prefixes), fitted on *training prefixes only*; or surface features (step and character counts, position), standardised with training statistics.
3. **Model.** L2-regularised logistic regression (or the MLP / GRU), target $y = 1$ if the final answer is wrong.
4. **Split.** Grouped by problem: 25% of problems held out (E3), or 5-fold cross-fitting grouped by problem, repeated with 5 seeds (Part 1). An automated test (`tests/test_no_leakage.py`) fails the build if any problem appears on both sides.

**Why grouped splits matter.** 3,400 rows contain only 2,842 problems. A random row split would put the same problem in training and test, and the detector would score well by memorising problems.

### 5.3 How the learned restart-point predictor (P4) is trained

Each step of a failed trace is one example with 8 features: PRM score and its drop, the generator's mean and maximum token negative log-likelihood (NLL) and entropy, relative position and step length. Label: 1 for the choke point chosen on half A of the continuations, 0 otherwise. Logistic regression on standardised features, trained on the *other* generators (for Llama: only on the three Qwen models), so the test model is never seen. At test time the step with the highest probability is the restart point.

### 5.4 How the learned choke-point localiser is trained (new)

- **Examples**: every step of the open failed traces of the training generators, nine features (the eight above plus "first PRM score below 0.5").
- **Soft labels**: instead of a hard 0/1 we use the posterior probability $P(d = j)$ that step $j$ is the choke point. Each step is entered twice, as label 1 with weight $P(d = j)$ and label 0 with weight $1 - P(d = j)$; weights are divided by the trace length so every trace counts once. This keeps sampling uncertainty in the target instead of pretending the choke point is known exactly.
- **Models**: logistic regression or gradient-boosted trees (scikit-learn).
- **Evaluation**: leave-one-generator-out. The held-out model is never used for training or for choosing hyperparameters.

**What we did not train, and why**

We did not fine-tune any LLM. The research question is about how existing models and verifiers behave; fine-tuning the generator would change $v_k$ itself. The hidden-state probes planned in the first version (E4) were not run, because others had already published that result and the GPU budget went to the counterfactual runs instead.

# 6 Hyperparameter tuning

**Project: Hyperparameter Tuning (5 marks)**

Detector hyperparameters were fixed in advance to avoid tuning on the test set, and checked for stability across seeds; measurement hyperparameters (threshold $\delta$, samples $m$, budget) were varied as sensitivity analyses; the models that needed tuning (learned localiser, judge threshold) were tuned with nested, group-wise cross-validation on data the test never touches.

## 6.1 Principle: tune only where the test set cannot leak in

Every choice made after looking at test results makes the result look better than it is. So the project uses three different treatments, depending on the role of the hyperparameter.

## 6.2 1. Fixed in advance, then checked for stability (detectors)

| Hyperparameter | Value (`configs/default.yaml`) |
|---|---|
| TF-IDF | uni+bigrams, max 20,000 features, min_df 2 |
| Logistic regression | L2, $C = 1.0$ |
| MLP | hidden layers 64 and 32, up to 500 iterations |
| GRU | hidden size 32, 200 full-batch epochs, Adam, learning rate 0.01 |
| Prefix fractions | 10, 20, 25, 30, 40, 50, 60, 70, 75, 80, 90, 100% |
| Seeds | 42, 1, 2, 3, 4 |

**Stability check instead of tuning.** The headline AUROCs vary by a standard deviation of only 0.003 (Q+G) and 0.006 (25% prefix) across the five seeds, so the conclusion does not hinge on a lucky split or setting.

## 6.3 2. Measurement hyperparameters: varied as sensitivity analyses

These define the measurement rather than a model, so we report results at several values instead of picking the best one.

- **Commitment threshold** $\delta$ (when a trace counts as "lost"): primary $1/8$, fixed before any data; $1/16$ and $1/4$ as sensitivity. Restart minus first-error backtracking keeps its sign at every value, e.g. Qwen2.5-1.5B +0.112 / +0.123 / +0.136 and Llama -0.011 / -0.022 / -0.031.
- **Continuations per prefix** $m$: split-half correlation of $v_k$ is 0.985 at $m = 64$ (Spearman–Brown projection 0.993), 0.951 at $m = 12$ and 0.896 at $m = 16$. More samples give more reliable curves; 16–64 is enough for the decisions we make, and the posterior check confirms the choke points are stable.
- **Budget** $b$: 1, 2, 4 and 8 restart attempts. Conclusions are reported at $b = 4$ and the whole curve is shown, so no single budget is cherry-picked.
- **Sampling temperature 0.7, top-$p$ 0.95**: fixed, standard values, the same for all models, so $v_k$ is comparable across models.
- **Token cap**: 1,536 (1.5B models) and 1,024 (larger). 17% and 29% of continuations hit the cap; re-running every analysis with those continuations treated as missing changes no conclusion.

## 6.4 3. Tuned with nested cross-validation (learned localiser)

The learned localiser has real hyperparameters, so we tuned them properly.

- **Search space** (24 configurations): feature set $\in$ {PRM features, generator features, all}; model $\in$ {logistic regression with $C \in \{0.01, 0.1, 1, 10\}$; gradient-boosted trees with depth $\in \{2, 3\}$ and learning rate $\in \{0.05, 0.2\}$, 200 trees}.
- **Nested, leave-one-generator-out.** Outer loop: hold out one model. Inner loop: on the other three models only, hold out each in turn, fit every configuration on the remaining two and score it on the held-out one. Pick the configuration with the best mean inner score, refit it on all three, and test once on the outer model.
- **Score**: posterior probability that the named step is the choke point.

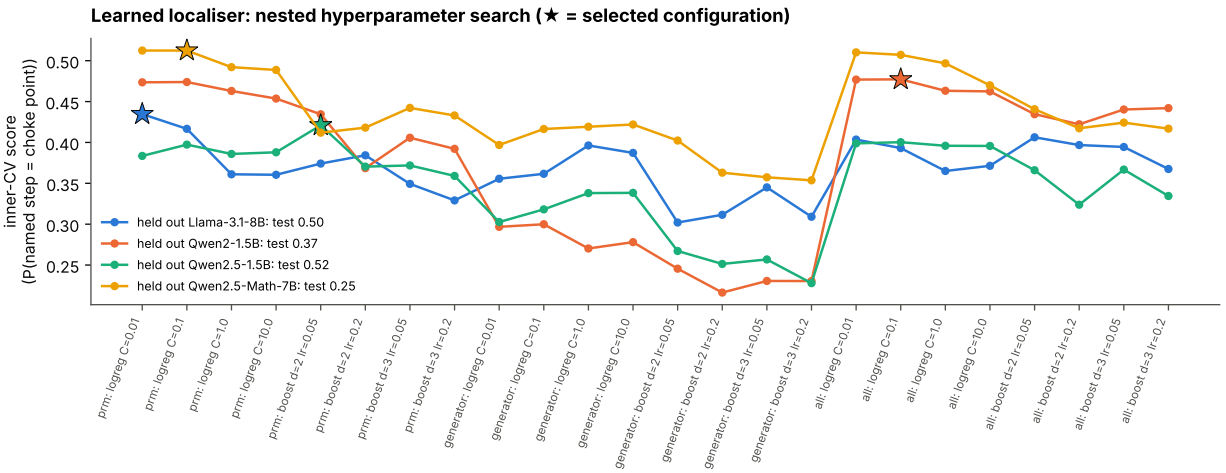



Figure 3: Nested hyperparameter search. One line per held-out model; stars mark the configuration selected on the inner folds; the legend gives the score on the held-out model.

| Held-out model | Selected features | Best inner score | Test score |
|---|---|---|---|
| Qwen2.5-1.5B | prm | 0.42 | 0.52 |
| Qwen2-1.5B | all | 0.48 | 0.37 |
| Qwen2.5-Math-7B | prm | 0.51 | 0.25 |
| Llama-3.1-8B | prm | 0.43 | 0.50 |

**What tuning taught us.** In three of the four outer folds the selection is a strongly regularised logistic regression ($C = 0.01$ or $0.1$), and in three of four it uses only the PRM features: there is little transferable signal, so the search prefers simple models. Inner scores do not carry over to the held-out model (Qwen2.5-Math-7B: 0.51 inside, 0.25 outside). The choke point is *model-specific*; a localiser tuned on some models does not transfer to a new one. Pooled, the tuned localiser finds the choke point with probability 0.39, no better than the fixed PRM rule (0.45).

### 6.5 4. A threshold tuned on DEV, tested on TEST (judge)

For Part 3 we needed a grader that never reads a LettuceDetect score. The LLM judge flags a response if it calls more than a fraction $f$ of its sentences unsupported. We chose $f$ from a grid fixed in advance (0 to 0.5) by Cohen's $\kappa$ with human labels on the 216 DEV drafts and tested on the 232 TEST drafts: $f^* = 25\%$, TEST $\kappa = 0.189$.

**The tuning mistake we caught**

The "dual consensus judge" added on 26 October had its threshold (0.3) chosen on *all* 449 labelled drafts; the DEV/TEST split was introduced afterwards and then called "out-of-sample". That is tuning on the test set. We withdrew the claim (Section 9). In the viva, this is a good example of why the split must come before the choice.

# 7 Results

**Research paper: Results (2.5 marks)**

Early warning reads difficulty (AUROC 0.588 from question + model); failures are often decided before step 1; the human error is the choke point about half the time; PRMs find errors (73%) not choke points (45%); restart is not beaten at matched compute (free oracle +0.028); the same in RAG (mechanism adds -0.000).

## 7.1 Mitigation: what works and what it costs

| RAG mitigation (RAGTruth, 450 sources) | cost | $E_1$ | $E_2$ | coverage |
|---|---|---|---|---|
| No mitigation | 1.00× | 80.9 | 46.0 | 100.0 |
| Delete flagged sentences | 1.02× | 79.6 | 13.1 | 99.8 |
| **Detect, restart, abstain (DRA)** | 1.36× | 76.7 | 12.9 | 98.9 |
| Claim repair, then delete | 1.71× | 80.0 | 18.9 | 99.8 |
| CARE (choke-point-aware) | 1.77× | 76.2 | 15.3 | 96.5 |
| Abstain on flagged QA | 1.02× | 67.3 | 10.4 | 87.1 |
| Chain-of-Verification (raw, $n$=192) | 3.75× | 97.4 | 72.4 | 100.0 |

Hallucination rates in %. In maths, plain restarting solves 21–36% of failed solutions with one attempt, 44–66% within four and 56–77% within eight. Human-scale prediction for DRA (Proposition 3): 50.2% → 26.6% [21.0, 32.2].

**Checking the law.** (1) Raising the retry budget from 1 to 4 on the same 450 sources raised coverage from 97.2% to 99.6% while delivered hallucination moved by only +0.6 ($E_2$) to +1.5 ($E_1$) points: retries buy coverage, not faithfulness, as predicted. (2) Sweeping the detector threshold gives an operating curve: 21.6% at 2.2 generations per answer (our setting) down to 12.7% at 4.4 with a stricter gate (predicted from human labels).

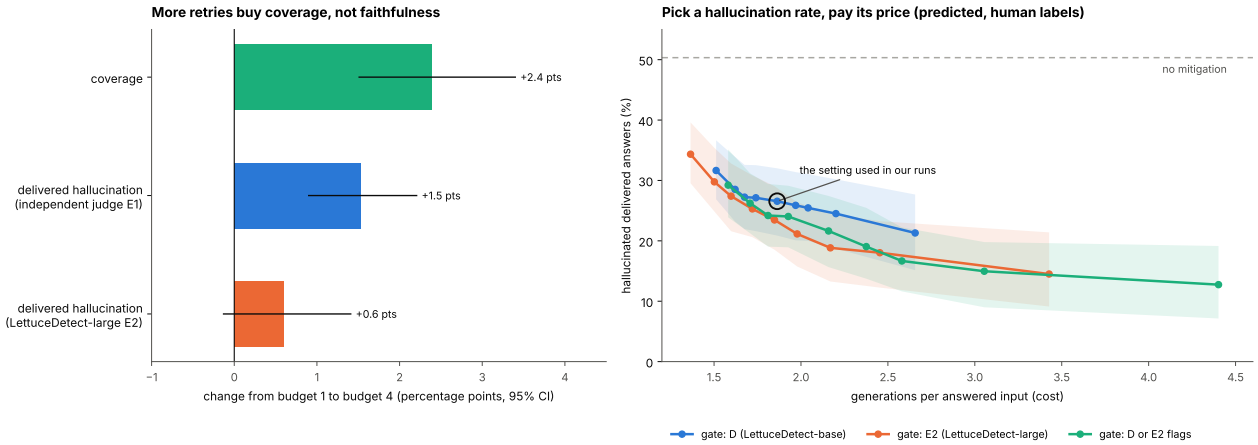



Figure 4: Checks of Proposition 3: budget invariance (left) and the predicted operating curve (right).

### 7.2 Part 1: early warning is a difficulty signal

| Detector (out-of-fold, 3,400 traces) | AUROC |
|---|---|
| Question only | 0.561 |
| Generator and tier only | 0.539 |
| Question + generator | 0.588 [0.570, 0.607] |
| First 25% of the reasoning only | 0.564 |
| Q + G + 25% of the reasoning | 0.593 (increment +0.005 [-0.004, +0.014]) |
| Q + G + the whole reasoning | 0.619 (increment +0.031) |

Within a problem, the 25% prefix ranks failing above succeeding traces 0.485 of the time (chance is 0.5). 23.5% of first errors are followed by a correct answer, and detectors rank these non-fatal errors as *riskier* than fatal ones (AUROC 0.456).

### 7.3 Part 2: where failures are decided

| | Qwen2.5-1.5B | Qwen2-1.5B | Qwen2.5-Math-7B | Llama-3.1-8B |
|---|---|---|---|---|
| Decided before step 1 | 33% | 52% | 66% | 58% |
| Human error = choke point | 0.647 | 0.438 | 0.423 | 0.633 |
| chance (position-matched) | 0.166 | 0.204 | 0.159 | 0.187 |
| Restart − backtrack to error, per attempt | +0.123 | +0.165 | +0.068 | -0.022 |

- Reliability: $v_k$ from independent batches correlates at $r = 0.985$ and 0.976; the most likely choke point has posterior probability $\geq 0.5$ in 94% (Qwen2.5-1.5B) to 88% (Llama) of traces.
- Preregistered on Llama: C3 (human error is the choke point more than chance) confirmed, Holm $p$ <0.002; C1 (restart beats backtracking to the error) *not* confirmed (-0.022 [-0.090, +0.047]). Replication on new batches (R1): +0.140 [+0.078, +0.202], confirmed.

### 7.4 Recovery policies at matched compute

| P(solved) − restart, budget 4 | Qwen2.5-1.5B | Qwen2-1.5B | Qwen2.5-Math-7B | Llama-3.1-8B |
|---|---|---|---|---|
| Restart: P(solved) | 0.657 | 0.514 | 0.437 | 0.492 |
| PRM largest drop | -0.651 | -0.514 | -0.022 | -0.063 |
| Learned restart point | -0.657 | -0.514 | -0.151 | -0.187 |
| Human first error (not deployable) | -0.047 | -0.075 | +0.039 | +0.001 |

The only positive estimate under full accounting: PRM largest drop on Qwen2.5-Math-7B at 8 attempts, +0.030 (exploratory).

### 7.5 ChokeBench

| Localiser | finds human error | finds choke point | Δ/attempt | Δ at 4×, free |
|---|---|---|---|---|
| Human first error | 1.00 | 0.51 | -0.066 | -0.045 |
| PRM: largest drop | 0.73 | 0.45 | -0.103 | -0.074 |
| PRM: first step $< 0.5$ | 0.68 | 0.38 | -0.077 | -0.062 |
| Learned, tuned | 0.68 | 0.39 | -0.085 | -0.071 |
| Generator: max NLL | 0.27 | 0.26 | -0.097 | -0.099 |
| Random step | 0.18 | 0.18 | -0.224 | -0.187 |
| Oracle choke point | – | – | +0.015 | +0.028 [+0.012, +0.046] |

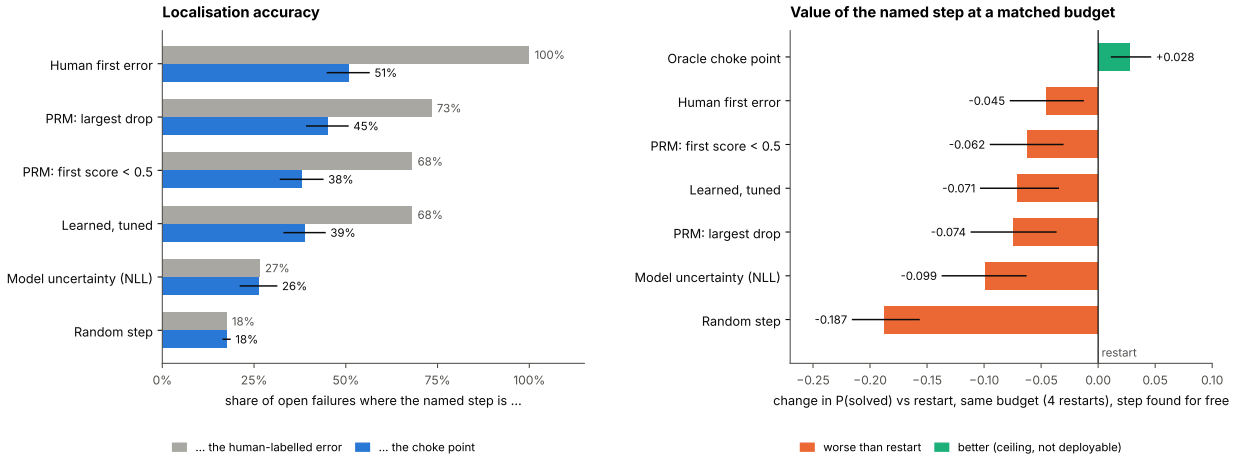



Figure 5: ChokeBench: localisation accuracy (left) and value of the named step at a matched budget (right).

### 7.6 Part 3: hallucination recovery

Preregistered (RAGTruth, judge $E_1$): CARE vs base -0.047 [-0.072, -0.025] (supported); CARE vs restart-until-$D$-passes -0.005 (equivalent within $\pm 0.03$; restart costs fewer tokens, 1,293 vs 1,677); CARE vs its restart-only version -0.000 [-0.005, +0.005]. Backtracking to just before the first human-marked hallucination vs restarting: +0.024 [-0.002, +0.049]. No snowballing (-0.006).

Five graders, held-out half of RAGTruth (hallucination rate, %):

| Policy | $E_1$ | $E_1$ calibrated | $E_2$ | dual | $D$ |
|---|---|---|---|---|---|
| Base model | 80.7 | 51.1 | 47.2 | 49.8 | 44.6 |
| Restart until $D$ passes | 79.0 | 49.8 | 14.6 | 18.9 | 0.9 |
| CARE | 77.7 | 46.8 | 19.3 | 24.9 | 2.1 |
| CoVe, revised answer | 71.4 | 43.9 | 41.8 | 42.9 | 30.6 |

The first two graders are independent of the gate $D$; the last three share its training data.

## 8 Result analysis: what the numbers mean

**Project: Result Analysis (5 marks)**

We read every result against a baseline, a null, an interval and a mechanism: Q+G for detectors, a position-matched null for choke points, restart at matched compute for policies, a free oracle as the ceiling, and an independent grader for RAG; and we explain the pattern with the supermartingale argument.

### 8.1 Reading 1: why early warning looked good and is not

A pooled AUROC of 0.588 sounds like the reasoning carries a warning signal. But hard problems fail more often for every model, and the question text alone reveals difficulty. When the comparison is *within* one problem, the signal disappears (0.485). The counterfactual data explain why: for 33–66% of failures the model could not solve the problem from scratch, so nothing in the reasoning could have changed the outcome.

### 8.2 Reading 2: errors are not choke points

The human-marked error is far more often the choke point than chance (e.g. Llama 0.633 vs 0.187), so labels are meaningful. But in the remaining traces either success had already collapsed earlier (latent commitment) or the trace was still recoverable after the error. And 23.5% of first errors do not lead to a wrong answer at all. "Where reasoning first goes wrong" and "where the outcome is decided" are different questions.

### 8.3 Reading 3: verifiers learn the label they are trained on

PRMs are trained and benchmarked on the human first error, and they find it well (73%). They find the choke point much less often (45%). Training a model directly for the choke point does not fix this (0.39), and its tuning shows why: the choke point depends on the generator, so features learned on some models do not transfer.

### 8.4 Reading 4: why restart wins

Three effects stack up.

1. **Theory.** Before the choke point a failed trace is, on average, worth no more than a fresh start (supermartingale). Resampling from earlier in a failed trace keeps the parts that already went wrong.
2. **The only real gain is saved tokens.** A later restart point means shorter continuations and therefore more attempts in the same budget, about 15–29% fewer tokens per attempt. That is why even the perfect oracle gains only +0.028.
3. **Verifiers cost compute.** A 7B PRM pass costs about five attempts of a 1.5B generator, so on small models every verifier-based policy runs out of budget.

### 8.5 Reading 5: hallucinations behave the same way, and graders can fool you

In RAG, resampling from before the first hallucinated sentence is not better than restarting, an online recoverability estimate adds nothing, and a hallucinated sentence does not make the next ones more likely. The more surprising lesson is about measurement. A loop that resamples until detector $D$ is satisfied will look excellent to any grader that resembles $D$ (LettuceDetect-large agrees with $D$ on 85.0% of drafts): the gate is grading itself. The independent judge sees a much smaller effect, but it agrees weakly with people, so neither number is the truth. Human ratings are needed, and the blinded kit is ready.

### 8.6 Reading 6: what to do in practice

The cheapest effective mitigation is to gate every output with a cheap detector, restart only what it flags, and abstain or delete flagged sentences when the budget runs out. Proposition 3 says where effort pays: the detector sets the hallucination rate of what is delivered, the budget sets how often the system answers, and the resampling point sets neither. So a better detector is worth more than a smarter repair.

**The big picture**

Across maths and RAG, "find the bad step and fix it" is a natural idea that does not survive a fair, compute-matched comparison against starting over. The useful levers are a better verifier (to know *whether* to retry) and an honest evaluator, not a smarter choice of *where* to retry.

## 9 Mistakes we caught and corrected

Being able to explain your own mistakes is a strength in a viva. These were found and fixed:

1. **Tie handling in AUROC** (original study): the lexical minus length-only gap at 25% fell from +0.073 to +0.052 ($p = 0.032$) after correcting how tied scores were ranked.
2. **Chat preambles** ("Sure, here's the rewritten sentence:") corrupted claim repairs; fixed in parsing before the full run.
3. **Chain-of-Verification echo**: Llama-2 copied the verification Q&A block into 89% of its answers, making CoVe look about 25 points worse than it is. Scoring only the revised answer fixes it.
4. **The dual consensus judge**: threshold chosen on all data, then called out-of-sample; and it shares training data with the gate. Withdrawn.
5. **Llama-3.1 "generalisation"**: reported detector flag rates as if they were failure rates; no recovery policy was run on those outputs. Withdrawn; Llama-3.1 RAG drafts are kept as data only.
6. **Paper build**: the committed PDF predated Part 3, the source did not compile, and six references had invented author lists. Rebuilt from the tables; references checked against arXiv and ACL pages.
7. **Human-evaluation blinding**: the A/B key had been committed publicly. Removed; future kits commit only its SHA-256 hash.

## 10 Limitations and next steps

- Open models up to 8B parameters; maths for Parts 1–2, one generator for RAG.
- ProcessBench solutions were re-segmented by its authors, so prefixes are near-, not exactly, on-policy.
- Llama-3.1-8B: 144 of 274 failed traces (GPU time limit); its RAG arm produced drafts only.
- The independent RAG graders agree weakly with people; no human rating of system outputs yet.
- ChokeBench and the learned localiser are exploratory, built from the same rollouts.

**Next:** (1) two human raters on the 80 blinded RAG pairs (regenerate the kit with a new seed first); (2) the same protocol on long-reasoning models such as R1-distilled or Qwen3, where earlier work finds prefixes can beat restarts, to find the boundary between the two regimes; (3) more continuations per prefix on Llama.

## 11 Viva questions with short answers

**Q1. What is new in your work?**
Measuring, by on-policy resampling, where failures are decided and comparing that with human error labels and PRMs at scale; ChokeBench; a compute-matched test of recovery policies with a theoretical ceiling; and the same test, preregistered, for RAG hallucinations.

**Q2. Why not just train a bigger detector?**
Because the limit is not the detector. Many failures are decided before step 1, and even a perfect choke-point oracle gains only +0.028.

**Q3. How do you know $v_k$ is not just noise?**
Independent sampling batches agree ($r = 0.985$), Bayesian posteriors put the choke point clearly on one step in most traces, and the choke point is chosen and scored on different halves of the samples.

**Q4. What is a position-matched null?**
The share of traces where a randomly placed "error", with the same position distribution as the real ones, would hit the choke point. It is about 0.16–0.20, far below the observed 0.44–0.65.

**Q5. Why group by problem when splitting?**
The same problem appears several times; a row split would leak it into the test set and reward memorisation.

**Q6. Which hyperparameters did you tune, and how?**
The learned localiser: 24 configurations, nested leave-one-generator-out CV. The judge threshold: grid on DEV, tested on TEST. Detector settings were fixed in advance and checked across 5 seeds; $\delta$, $m$ and budget are reported as sensitivity.

**Q7. Why did the tuned model not beat the simple PRM rule?**
The choke point is model-specific: the best inner score (0.51) dropped to 0.25 on the held-out model. Little transferable signal means regularised, simple models win and still do not beat a fixed rule.

**Q8. What does "matched compute" include?**
Every prefill and generated token, PRM passes, probes and oracle rollouts, weighted by model size, with the same budget for every policy.

**Q9. Explain the supermartingale argument simply.**
On average over failures, the chance of success at any point you pick without looking ahead is not higher than at the start; so backtracking cannot beat restart per attempt unless it looks ahead or saves tokens.

**Q10. Which results failed?**
C1 and C2 on Llama (backtracking to the human error is not worse than restart for Llama), and every preregistered CARE comparison against an alternative strategy.

**Q11. Why is a negative result valuable?**
It tells the field which baseline to beat. "LLMs cannot self-correct reasoning yet" is a well-known example of a negative result that changed practice.

**Q12. Why do graders disagree in Part 3?**
Restart-until-$D$-passes selects outputs that $D$ likes; a grader trained on the same data likes them too. An independent grader sees much less.

**Q13. What is preregistration and why did you do it?**
Writing hypotheses and analyses into git before collecting data, so that results cannot be selected after the fact. Our commits `a6fb04c` and `459ff22` predate the data they test.

**Q14. What would you do with more compute?**
Human ratings for Part 3, and the protocol on long reasoning models, to see where prefixes start to beat restarts.

**Q15. So does your research reduce hallucination?**
Yes. Detect–restart–abstain reduced hallucination significantly in a preregistered test (-0.047, $p = 0.0003$), is the cheapest resampling mitigation (1.36× the base cost), and is predicted to cut hallucinated answers from 50.2% to 26.6% for people. Our contribution is to show that this simple recipe is the right one, prove what limits it, and show that the more complex surgical alternatives do not beat it.

**Q16. State and explain Proposition 3.**
The loop returns the first answer the detector passes. Each draw is passed-and-hallucinated with probability $h\beta$ or passed-and-faithful with $(1 - h)(1 - \alpha)$, so a returned answer is hallucinated with

probability $h\beta/(h\beta + (1-h)(1-\alpha))$ whatever the number of retries. Retries raise coverage, only the detector lowers hallucination.

**Q17. How did you test your own law?**

It makes a falsifiable prediction: more retries should raise coverage but not change the hallucination rate of delivered answers. From budget 1 to 4, coverage rose by +2.4 points and delivered hallucination moved by +0.6 to +1.5 points. A large change would have refuted it.

**Q18. Can you beat Perplexity?**

Not claimed. Perplexity is a web-search system judged on open-domain factuality (SimpleQA); our task is faithfulness to given sources. The fair test is to run its API on the same RAGTruth items and score it with the same graders, which we list as future work.

**Q19. How can someone reproduce your numbers?**

`make story` regenerates ChokeBench, the grader analysis, the figures, every paper number and the PDF on a CPU in a few minutes; the GPU runs have resumable Kaggle scripts.